# Ders 4 Genişletilmiş Notu: Düzenli İfadeler ve Biçimbilimsel Analiz

## Ders 4 Genişletilmiş Notu: Düzenli İfadeler ve Biçimbilimsel Analiz

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Düzenli Dillerin Yeri
- 2. Kleene Operatörleri
- 3. Düzenli İfadelerin Otomat Karşılıkları
- 4. Adım Adım Bir Düzenli İfade Kurmak
- 5. Düzenli İfadeler Nerede Kullanılır
- 6. Temel Komutlar
- 7. Bir Uygulama Problemi: Adresler
- 8. Biçimbilimsel Analiz ve Biçimbirim
- 9. Ek Türleri ve Kelime Oluşturma
- 10. İngilizce ile Türkçe Arasındaki Fark
- 11. Çoklu Çözüm ve Anlam Belirsizliği
- 12. Biçimbilimsel Analiz Nerede Kullanılır
- 13. Bağlılık Analizi
- 14. İngilizcede Çekim ve Yapım Ekleri
- 15. Biçimbilimsel Çözümleyicinin Bileşenleri
- 16. İngilizce İçin Sonlu Durum Otomatları
- 17. Kök Bulma: Soldan ve Sağdan
- 18. Türkçe Çözümleyicinin Akışı
- 19. Türkçe Ek Otomatları


### 1. Düzenli Dillerin Yeri

Bu ders iki büyük konudan oluşuyor. Birinci yarıda düzenli ifadeler (regular expression, kısaca regex), ikinci yarıda biçimbilimsel analiz (morphological analysis) anlatılıyor. İkisi birbirine bağlı. Biçimbilimsel çözümleyiciler sonlu durum otomatlarıyla kurulur. Düzenli ifadeler de aynı otomat mantığının kısa bir yazımıdır.

Hoca önce önceki dersteki dil sınıflandırmasını hatırlattı. Diller dört sınıfa ayrılmıştı:

- **Tür 0:** En dıştaki ve en geniş dil. Kısıtlama yoktur. Türetme kurallarının sol ve sağ tarafındaki uç değişken ve söz dizimsel değişken sayısında sınır yoktur.
- **Tür 1:** İlk kısıt burada gelir. Türetme kuralının sol tarafındaki eleman sayısı, sağ tarafındaki eleman sayısına eşit veya ondan küçük olmalıdır.
- **Tür 2:** Bağlamdan bağımsız (context-free) diller. Programlama dilleri bu sınıftandır. Bu yüzden programlama dillerinde kalıbın dışına çıkılamaz, hemen sözdizimi hatası (syntax error) alınır.
- **Tür 3:** Düzenli diller. En kısıtlı dildir. Kurallar çoktur ve dışına çıkılamaz. Yine de bu dille çok iş yapılabilir.

Elemanlar iki türlüdür. Uç değişkenler cümleyi oluşturan kelimelerdir, yani dilin sözlüğündeki (lexicon) kelimeler. Söz dizimsel değişkenler ise dilin kalıplarıdır. Türkçede özne-nesne-yüklem veya özne-belirtili nesne-dolaylı tümleç-yüklem bir kalıptır. Bir sıfat tamlaması için sıfat-sıfat-sıfat-isim de bir kalıptır.

Düzenli dillerin de bir alfabesi vardır. Düzenli diller arka arkaya bağlanabilir (concatenation). Üzerlerinde birleşim ve kesişim işlemleri yapılabilir. Bir de Kleene operatörleri uygulanır.

## 2. Kleene Operatörleri

Yüksek lisans öğrencileri bu operatörleri derleyici (compiler) veya otomat teorisi dersinden hatırlar. İki operatör vardır: yıldız (*) ve artı (+). Aralarındaki tek fark, hiç geçmeme durumuna izin verip vermemeleridir.

- `a*` ifadesi boş dizgeyi (λ) ve ardışık yazılan bütün dizgeleri verir: λ, a, aa, aaa, …
- `a+` ifadesinde “a” en az bir kez geçer: a, aa, aaa, …

Hoca bu operatörlerin hem komut hem sayaç olarak düşünülebileceğini söyledi. Bir `for` döngüsü gibi düşünülür: * sıfırdan n’ye, + birden n’ye kadar sayar.

Adlarını, ortaya atan Stephen Kleene’den alırlar. Düzenli ifade aslında belirlenimci olmayan bir sonlu otomattır (non-deterministic finite automaton). Problem çözülürken belirlenimci sonlu otomata (deterministic) dönüştürülür.

## 3. Düzenli İfadelerin Otomat Karşılıkları

Düzenli ifade yazarken otomat mantığını düşünmek gerekir. Temel durumlar şunlardır.

**Durum geçişi.** Bir durumdan (state) başka bir duruma geçilir. Örneğin gelen kelimenin isim olup olmadığı kontrol edilir. İsimse sonraki duruma geçilir. Bu yapı, önceki derste görülen genişletilmiş geçiş ağlarında da vardı.

**Epsilon (ε) geçişi.** Hiçbir girdi olmadan durum değiştirilir.

**Boş küme.** İki durum birbirine hiç bağlanamaz.

**Birleştirme (concatenation).** Birinci dilin çıktısı ikinci dilin girdisine girer. Hiçbir şart yoktur, ifadeler arka arkaya kullanılır.

**Veya (|, pipe).** İki dil vardır ve başlangıç noktası ortaktır. Başlangıçtan ε ile iki dile de gidilebilir. Biri seçilir ve işletilir. Bitiş durumuna (final state) ulaşılamazsa yanlış seçim yapılmış demektir, diğeri denenir. İkisinin çıktısı, onları birleştiren otomatın ortak bitiş durumuna bağlanır.

**Kleene yıldızı.** Ortada bir dil (hoca buna L1 dedi) vardır. Üç durum düşünülür:

- **Hiç çalışmama:** Başlangıçtan ε ile doğrudan çıkışa geçilir.
- **Bir kez çalışma:** Dil işlenir, bitiş durumuna gelinir ve çıkılır.
- **Birden çok kez çalışma:** Bitiş durumundan ε ile yeniden başlangıca dönülür. Dil yeniden işlenir. Döngü bu dönüş üzerinden sağlanır.

Düzenli ifade yazmanın özü budur: ifadeler arasına `|` konabilir, ifadeler arka arkaya bağlanabilir ya da bir ifade defalarca çalıştırılabilir.

## 4. Adım Adım Bir Düzenli İfade Kurmak

Hoca tablette küçük bir örnekle ilerledi. Burada düzenli ifade kurulan otomattır, dizgeler ise otomata verilen girdilerdir. Aşağıdaki sonuçlar dizgenin tamamının ifadeye uyması anlamındadır.

**Adım 1.** İfade `abc` yazıldı. “abc” girdisi çalışır. “bc” girdisi çalışmaz, çünkü “a” zorunludur.

**Adım 2.** “bc” de tanınsın diye “a”nın üstüne yıldız kondu: `a*bc`. “a” artık olmayabilir. Ama “ac” girdisi çalışmaz, çünkü bu kez “b” zorunludur.

**Adım 3.** “b”nin üstüne de yıldız kondu: `a*b*c`.

- “abc”: “a” bir kez, “b” bir kez, sonra “c”. Çalışır.
- “bc”: “a” alınmaz, “b” bir kez, sonra “c”. Çalışır.
- “ac”: “a” bir kez, “b” alınmaz, sonra “c”. Çalışır.
- “aabc”: `a*` iki kez döner, “b” bir kez, sonra “c”. Çalışır.

**Adım 4.** “acbc” girdisi geldi. `a*b*c` bunu sağlamaz. Çözüm, ifadenin dışına bir yıldız daha koymaktır: `(a*b*c)*`. Döngünün ilk turunda “a” alınır, “b” alınmaz, “c” alınır (“ac”). İkinci turda “a” alınmaz, “b” ve “c” alınır (“bc”).

Burada “c”nin yıldızsız olması, parantezin içine her girişte “c”nin mutlaka gelmesi gerektiğini söyler. Dışarıda * yerine + olsaydı parantezin içine en az bir kez girmek zorunlu olurdu.

Bu ayrıntının neden önemli olduğunu hoca şöyle açıkladı: Gerçek dünyada a, b, c yerine kelimeler veya ifadeler gelir. Bir belgede aranan kalıplar tekrar edebilir veya etmeyebilir. Probleme göre operatör seçilir. Bu operatörler çok farklı varyasyonları yakalama imkânı verir.

## 5. Düzenli İfadeler Nerede Kullanılır

**Arama motorları ve bilgiye erişim.** Aranan şeyin çok farklı biçimleri olabilir. Hoca kendi adını örnek verdi: Aynı ada sahip başka biri de var. Yalnız akademik çalışmalara ulaşılmak isteniyorsa adın önünde mutlaka bir unvan aranır. Unvan büyük veya küçük “d” ile, ya da başka bir unvan olarak yazılmış olabilir. Ad yazılmayıp yalnız soyadı yazılmış da olabilir. Adın olup olmaması yıldıza karşılık gelir. Bilgiye erişim sonucunda gelen dokümanların üzerinde böyle bir ifade çalıştırılarak istenen cümlelere ulaşılır.

**Kelime işleme (word processing).** Bazı kelimeler yanlış yazılmıştır. Büyük-küçük harf farkları olabilir. Türkçe karakter kullanılmamış olabilir (“küçük”, “kucuk” gibi). Bütün bu biçimleri getiren tek bir ifade yazılabilir.

**Derlemde kelime sıklığı saymak.** Derlem (corpus) dokümanlar topluluğudur. “okul”, “okula”, “okulda”, “okulum” gibi biçimlerin hepsi tek kelime olarak sayılmak isteniyorsa kök bulmakla uğraşmaya gerek yoktur. “okul ile başlayan, sonuna her türlü ek alabilen” diye bir ifade yeterlidir.

**Veri doğrulama (data validation).** TC kimlik numarası, telefon numarası veya e-posta adresinin doğru biçimde girilip girilmediği kontrol edilir. Yanlışsa kişiye uyarı verilir.

**Nasıl yazıldığı hatırlanmayan kelimeyi bulmak.** “scholarships” gibi bir kelimede bazı harflerin yeri unutulmuş olabilir. Emin olunan karakterler verilir, diğer yerler esnek bırakılır.

**Belirli bir başlangıcı olan her şeyi bulmak.** Bütün HTTP adresleri bulunmak istenirse “başında bu ifade olan, gerisi ne olursa olsun” diye bir ifade yazılır.

**Araçlar.** Düzenli ifadeler şu ortamlarda kullanılır:

- vi, Emacs, Kate gibi metin düzenleyiciler (genelde Unix tabanlı).
- grep, sed gibi konsol uygulamaları.
- Perl, Python, PHP, JavaScript programlama dilleri.
- Eclipse, Visual Studio gibi geliştirme ortamları.

  **Hocanın vurgusu:** Düzenli ifadeler büyük-küçük harfe duyarlıdır (case sensitive). “İçinde a geçen kelimeleri getir” denirse “okula” gelir ama “Ali” gelmez, çünkü “Ali” büyük “A” ile yazılmıştır. Bunu unutmak çok hata yaptırır.

Bir dizgede kalıbın ardışık sırada geçmesi yeterlidir. “okul” arandığında kalıbın öncesinde ve sonrasında başka karakterler olabilir, “okula” da gelir.

> **Hocanın vurgusu:** Derste 10’a yakın komut gösterilecek. Bu komutlarla yapılabilecek işlerin yaklaşık %80’i yapılabilir. Regex bir dil gibi oturup ezberlenmez, ihtiyaç oldukça kullanılır. Özel bir durumda kaynaklara bakılır.

## 6. Temel Komutlar

### 6.1 Başlangıç ve Bitiş: ^ ve $

^ (şapka) bir şeyin başlangıcını, $ (dolar) sonunu gösterir.

- `^Banu`: “Banu” ile başlayan ifadeler. “Banu”, “Banu’yu”, “Banu’da”, “Banu’nun” gelir. Küçük “b” ile yazılanlar gelmez.
- `Banu$`: “Banu” ile bitenler. Önünde başka karakterler olması önemli değildir.
- `^Banu$`: Yalın “Banu”. Kelime “Banu” ile başlar ve “Banu” ile biter.
- `^$`: Arada hiçbir şey yoksa boş satırlar bulunur.

Örnek olarak “gül”, “güler”, “songül” kelimeleri denendi:

| İfade | gül | güler | songül |
|---|---|---|---|
| `^gül` | bulur | bulur | bulmaz |
| `gül$` | bulur | bulmaz | bulur |
| `^gül$` | bulur | bulmaz | bulmaz |

### 6.2 Karakter Sınıfı: `[ ]`

Köşeli parantez bir karakter sınıfı belirtir. İçindeki karakterlerden biri gelir. Aralarında “veya” vardır.

**Makina ve makine.** Kelimenin doğrusu “makine” olsa da “makina” diye de yazılıyor. İkisi birden isteniyorsa `makin[ae]` yazılır. “makin”den sonra gelen karakter “a” veya “e” olur.

**`[ds]e[lb]i` örneği.** Birinci karakter “d” veya “s”, ikinci karakter mutlaka “e”, üçüncü karakter “l” veya “b”, son karakter “i”dir. Bu kalıp “deli”, “seli”, “debi”, “sebi” dizgelerini tanır.

- “sebil” ve “delil” de bulunur. Sondaki “l” kalıpta yok ama önemli değil. Aranan şey bu kalıbı içeren kelimelerdir.
- “Salim” bulunmaz. İkinci karakter mutlaka “e” olmalıdır, burada “a” var.

**Büyük-küçük harf.** Bütün “Banu”lar için `[Bb]anu` yazılır.

**Aralık.** HTML belgesindeki bütün başlıklar (header) bulunmak istensin. Altı seviye vardır. Tek tek ifade yazmak yerine `h[123456]` yazılabilir. Yüz seviye olsaydı sayıları tek tek yazmak saçma olurdu. Tire aralık belirtir: `h[1-6]`. Bir ve altı dahildir. Diğer örnekler:

- `[0-9]`: 0 ve 9 dahil arasındaki rakamlar.
- `[a-z]` ve `[A-Z]`: ASCII tablosundaki küçük ve büyük harf aralıkları.
- `[a-zA-Z]`: Küçük veya büyük harf.

### 6.3 Olumsuzlama: `[^ ]` ve Şapkanın Üç Kullanımı

Köşeli parantezin içinde başta gelen ^, kendisinden sonrakilerin dışındakiler anlamına gelir. Mantıksal “değil” gibi düşünülür. İstenmeyen şeyler şapkadan sonra yazılır.

`[^5-8]` ifadesi 5 ile 8 dışındaki karakterleri tanır.

> **Ders sonrası not:** Hoca bu ifade için örnek olarak 1, 2, 3, 4, 9 rakamlarını saydı. İfade rakamla sınırlı değildir. 0, harfler ve boşluk da dahil 5, 6, 7, 8 dışındaki her karakter eşleşir.

**`Türkçe[^d]` örneği.** “Türkçe”den sonra gelen karakter “d” dışında bir karakter olmalıdır.

- “Türkçemiz” bulunur: “e”den sonra “m” gelmiş.
- “Türkçeyi” bulunur: “e”den sonra “y” gelmiş.
- “Türkçede” bulunmaz: “e”den sonra “d” gelmiş.
- “Türkçe” bulunur: “e”den sonra bir şey yok gibi görünür ama boşluk gelmiştir. Boşluk da bir karakterdir.
- “TürkçeD” bulunur: Yalnız küçük “d” dışlanmıştır.

> **Ders sonrası not:** Metin “Türkçe” ile bitiyorsa, yani arkasında hiçbir karakter yoksa, bu ifade eşleşmez. `[^d]` bir karakterin varlığını gerektirir.

Diğer örnekler: `[^A-Z]` büyük harf dışındaki karakterlerdir. `[^\.]` nokta dışındaki karakterlerdir.

Şapka üç şekilde karşımıza çıkar:

- **Köşeli parantez içinde, başta:** Olumsuzlama. `[^e]`, "e" dışındaki karakterler.
- **Köşeli parantez dışında, başta:** Başlangıç. `^gül`.
- **Köşeli parantez içinde, başta değil:** Karakterin kendisi. `[e^]`, "e" veya "^" karakteri.

  **Ders sonrası not:** Hoca `[a^c]` için "a veya c'nin dışındaki her şey" dedi ve böyle bir yazımın anlamsız olduğunu söyledi. Doğrusu: `^` köşeli parantezin başında olmadığı için karakterin kendisidir. `[a^c]` ifadesi "a", "^" veya "c" ile eşleşir.

  **Ders sonrası not:** Hoca köşeli parantez dışındaki `a^b` ifadesinin "a^b" kalıbını aradığını söyledi. Python'da denendiğinde `a^b` ifadesi "a^b" metninde eşleşmez, çünkü `^` burada başlangıç çapasıdır. Bu metni aramak için `a\^b` yazılır.

### 6.4 Joker `.` ve Kaçış Karakteri `\`

Tek başına `.` joker karakterdir (wildcard). Yerine herhangi bir karakter gelebilir.

`sıc.k` ifadesi "sıcak" ve "sıcık" dizgelerini bulur. "Sıcak" bulunmaz, çünkü ifade küçük "s" ile başlıyor.

Bazı semboller komut olduğu için kendileri aranmak istendiğinde önlerine ters eğik çizgi konur. `\.` nokta karakterinin kendisidir. `\?` soru işaretinin kendisidir.

**IP örneği.** Bir IP adresiyle başlayan ve sonunda herhangi bir karakter daha olan dizgeler aransın: `193\.140\.4\.1.`

- 193 yazılır. Noktanın joker olmadığını göstermek için `\.` yazılır.
- 140, `\.`, 4, `\.`, 1 yazılır.
- En sondaki nokta jokerdir. Oraya 3, A veya boşluk gelebilir.

**`.*` yapısı.** Joker bir karakter getirir. Yıldız da bunu tekrar edebilir. Böylece bir karakter, bir karakter dizisi, hatta boşlukla ayrılmış birkaç kelime yakalanır.

`YTÜ.*kulüpleri` ifadesi "YTÜ kulüpleri", "YTÜ öğrenci kulüpleri", "YTÜ kız öğrenci kulüpleri", "YTÜ gözlüklü kız öğrenci kulüpleri" dizgelerinin hepsini bulur. Dokümanlarda bütün kulüpleri tek tek aramaya gerek kalmaz. Hoca bu örneği en sevdiği örneklerden biri olarak andı.

### 6.5 Seçimlik: `?`

`?` kendisinden önce gelen karakterin seçimlik olduğunu belirtir. O karakter olabilir de olmayabilir de.

- Sonuna seçimlik "i" konan "evler" kalıbı "evler" ve "evleri" dizgelerini bulur.
- `colou?r`: İngiliz İngilizcesindeki "colour" ve Amerikan İngilizcesindeki "color". Değişen yer yalnızca "u" harfidir. Soru işareti "u"nun arkasına konur.

### 6.6 Yıldız ve Artı

`ab*c` ifadesinde "a" ile başlanır, "c" ile bitilir. Ortada "b" olmayabilir ya da defalarca tekrar edebilir. "abd" bulunmaz, çünkü "c" zorunludur.

**Soru:** Beş tane "a" gelip sonra "c" gelse, arada "b" de olmasa bulur mu?

**Cevap:** Hoca hayır dedi. "a" yalnız bir kez yazılabilir, sonrasında yıldız nedeniyle yalnız "b" tekrar edebilir. Birden çok "a" için ifadenin dışında bir döngü gerekir.

> **Ders sonrası not:** Hocanın cevabı dizgenin tamamı ifadeyle eşleşecekse doğrudur. Bu derste anlatılan "kalıbı içeren dizgeleri bulma" biçiminde ise `ab*c` ifadesi "aaaaac" dizgesini bulur, çünkü dizgenin sonunda "ac" parçası vardır. Python'da `re.search` ile denendiğinde eşleşti, `^ab*c$` ile eşleşmedi.
>
> **Ek bilgi (derste anlatılmadı):** Python'da `re.search` kalıbı dizgenin herhangi bir yerinde arar. `re.fullmatch` ise dizgenin tamamının kalıba uymasını ister. Bir ifadenin bir dizgeyi "tanıyıp tanımadığı" sorulurken hangisinin kastedildiğine dikkat etmek gerekir.

**`[0-9][0-9]*` örneği.** İlk köşeli parantez yıldızsız olduğu için en az bir rakam zorunludur. Sonrasında yıldız nedeniyle ikinci, üçüncü, dördüncü basamak gelebilir veya gelmeyebilir.

**Soru:** İkinci `[0-9]` neden yazıldı? Niye yalnız `[0-9]*` yazılmadı?

**Cevap:** `[0-9]*` yazılsaydı hiç rakam olmama durumu da kabul edilirdi. İlk parça, bu durumu ortadan kaldırıp en az bir rakamı zorunlu kılar.

Bir öğrenci + kullanmanın daha pratik olduğunu söyledi. Hoca onayladı: `[0-9]+` aynı işi yapar ve daha kısadır. Uzun hâl yalnız komutları göstermek için verilmişti.

> **Hocanın vurgusu:** Aynı işi yapan iki ifadeden kısa ve sade olanı tercih edilmeli. Yarım satırlık ifade ile iki satırlık ifade aynı sonucu verebilir. Ama bir yıl sonra yeni bir durum eklemek gerektiğinde karmaşık bir ifadeyi anlamak çok zorlaşır. Sıfırdan yazmak daha kolay gelebilir. Basit bir ifadeye yenilik eklemek ise pratiktir.

### 6.7 Kelime Sınırı: `\b`

`\b` özel bir komuttur. Aranan ifadenin önünde veya arkasında sınırlayıcı olmasını sağlar. `\beli\b` ifadesi önünde ve arkasında sınır olan "eli"yi bulur.

Hoca bu örnekle ilgili bir anısını anlattı. Bu ifadeyi ilk gördüğünde uzun süre "beli" diye okumuş, baştaki `\b`nin ayrı bir komut olduğunu çok sonra fark etmiş.

`\B` ise sınır olmayan durumu anlatır.

> **Ders sonrası not:** Hoca, sınırlayıcının boşluk veya tırnak ya da yıldız olabileceğini, örneğin sınır yıldızsa "eli"nin iki yanında yıldız olması gerektiğini söyledi. `\b` belirli bir karakter değildir. Kelime karakteri ile kelime karakteri olmayan bir şey (boşluk, noktalama, satır başı veya sonu) arasındaki sıfır genişlikli konumdur. Belirli bir karakter sınır olarak isteniyorsa kalıba açıkça yazılmalıdır.
>
> **Ders sonrası not:** `\B` "esnek, her şey olabilir" demek değildir. O konumun kelime sınırı olmamasını ister. Örneğin `\Beli` ifadesi "beli" içinde eşleşir, tek başına "eli" kelimesinde eşleşmez.

**Soru:** Metinde `\b` dizgesinin kendisi aranırsa programlamadaki gibi ters eğik çizgi iki kez mi yazılmalı?

**Cevap:** Hoca hiç denemediğini, bilmediğini söyledi. Python'daki düzenli ifade kütüphanesiyle veya internetteki regex deneme siteleriyle denenebileceğini ekledi. Soruyu soran öğrenci Python'da denedi: Ters eğik çizgi iki kez yazılınca çalıştı. Hoca, bir konuyu denemeden bilmediğini söylemenin doğal olduğunu özellikle belirtti. Regex uç durumlarda denenerek öğrenilir.

> **Ek bilgi (derste anlatılmadı):** Python dizgelerinde `\` zaten kaçış karakteridir. Bu yüzden düzenli ifadeler genellikle ham dizgeyle (`r"\bkelime\b"`) yazılır. Böylece ters eğik çizgiler Python tarafından yorumlanmadan regex motoruna ulaşır.

Boşluk aramak için `\s` kullanılır. Hoca bu noktayı öğrencinin denemesi üzerine anlattı.

### 6.8 Veya: `|`

Bu yıl dördüncüsü düzenlenen bütün sempozyumlar bulunmak istensin.

- Sıra sayısı yazıyla ("Dördüncü", "dördüncü") veya rakamla ("4.") yazılabilir. Rakamdan sonraki nokta için `\.` kullanılır.
- Arada sempozyumun adı gelir. NLP olabilir, görüntü işleme olabilir, üç dört kelimelik bir ad olabilir. Bunu `.*` sağlar.
- Sonda "Sempozyum" veya "sempozyum" gelir.

Genel biçim şöyledir:

```
([Dd]ördüncü|4\.).*[Ss]empozyum
```

Dördüncüsü değil, kaçıncı olursa olsun bütün sempozyumlar istensin. Hoca bu durumda "4." yerine köşeli parantezle 1-99 arası bir sayı ve nokta yazılabileceğini söyledi. Sona da sempozyum veya konferans seçeneği eklenir: `([Ss]empozyum|[Kk]onferans)`.

**Ders sonrası not:** Köşeli parantez tek karakter tanır. `[1-99]` ifadesi 1'den 99'a kadar sayıları değil, yalnız 1-9 arasındaki tek rakamı tanır. 1-99 arası sayı için `[1-9][0-9]?` yazılabilir. Örneğin `[1-9][0-9]?\..*([Ss]empozyum|[Kk]onferans)` ifadesi "10. Robotik Konferansı" dizgesini buldu.

### 6.9 Sayaçlar

Üç tür sayaç vardır:

- `{n}`: Kendinden önceki karakter tam n kez ardışık gelmeli.
- `{n,m}`: En az n, en çok m kez.
- `{n,}`: En az n kez, üst sınır yok.

**"Ateşoğlu" örneği.** Yalnız bu kelime aranacaksa regex gerekmez, metin düzenleyicide arama yeter. Ama "önünde dört karakter, arkasından üç rakam, sonra Ateşoğlu" gibi bir kalıp isteniyorsa sayaçlar gerekir. Rakam kısmı `[0-9]{3}` olur. Örneğin "ABCD129Ateşoğlu" bu kalıba uyar.

**"Yılmaz" örneği.** Bir dokümanda önce bölüm kodu (BLM, EHM gibi üç dört harf), sonra öğrenci numarası, sonra soyadı geçiyor olsun. Soyadı "Yılmaz" olan bütün öğrenciler tek bir ifadeyle bulunabilir.

Soru işareti de sayaç gibi düşünülebilir: Önündeki karakteri sıfırdan bire kadar sayar. Artı birden n'ye, yıldız sıfırdan n'ye sayar.

### 6.10 İşlem Önceliği

Matematikteki işlem önceliği gibi bir sıra vardır. Yüksekten düşüğe:

1. Parantezler.
2. Sayaçlar: `*`, `+`, `?`, `{ }`.
3. Seriler ve bağlayıcılar: `^`, `$`.
4. Veya: `|`.

### 6.11 Özel Operatörler

Hoca en çok kullanılanları listeledi:

- `\t`: Sekme karakteri.
- `\n`: Yeni satır karakteri.
- `\?`, `\*`, `\.`: Karakterlerin kendisi.
- `\s`: Boşluk. `\S`: Boşluk olmayan karakter.
- `\w`: Kelime karakteri. `\W`: Kelime karakteri olmayan.
- `\d`: Herhangi bir rakam. `\D`: Rakam olmayan karakter.

  **Ders sonrası not:** Hoca `\w` için "alfanümerik veya boşluk karakteri" dedi. Doğrusu: `\w` harf, rakam ve alt çizgi ile eşleşir, boşlukla eşleşmez. `\W` bunların dışındaki her karakterdir ve boşluk da buna dahildir.

## 7. Bir Uygulama Problemi: Adresler

Hoca, komutların hepsinin birlikte kullanıldığı gerçekçi bir problem olarak farklı biçimlerde yazılmış adresleri verdi. Aynı bilgi çok farklı yazılabilir:

- Sokak için "Sokak", "S.", "Sk.", "Sok."
- Daire için "Daire 5", "D:5", "D 5"

Amaç, bir metindeki adreslerin olabildiğince çoğunu tek bir ifadeyle bulmaktır. Hocaya göre bu problemi çözen biri, gösterilen komutların hepsini öyle ya da böyle kullanmak zorunda kalır ve konuyu bitirmiş olur.

## 8. Biçimbilimsel Analiz ve Biçimbirim

Ara sonrasında ikinci konuya geçildi. Hoca önce adlandırmayı açıkladı: Kaynaklarda “biçimbilimsel” ve “biçimbirimsel” ikisi de kullanılıyor.

**Tanım.** Biçimbilimsel analiz, bir metindeki kelimenin bilgisayar tarafından otomatik olarak çözümlenmesidir. Kelime kök ve eklerine ayrılır ve görevleri belirlenir. Görev, kelimenin veya ekin tipi ve hangi etiketle anıldığı bilgisidir.

**Biçimbirim (morpheme).** Kök ve eklere biçimbirim denir. Kelimedeki en küçük anlamlı birimdir.

- “fox”: Bir biçimbirim.
- “cats”: İki biçimbirim. “cat” kök, “s” çoğul eki.
- “çiçekler”: İki biçimbirim.

**Kök ve gövde (stem).** “gözlük” kelimesinin kökü “göz”den gelir. Ama “gözlük”, gözden türetilmiş bir kelimedir, sözlükte ayrı bir anlamı vardır ve bir nesneyi anlatır. Bu yüzden gövdesi “gözlük” olarak alınır. “göz”e kadar inilirse nesne anlamı kaybolur. Metinde gözler sayılacaksa gözlük sayılmamalıdır, çünkü o göz değildir.

## 9. Ek Türleri ve Kelime Oluşturma

Ekler (affix) konumlarına göre ayrılır:

- **Ön ek (prefix):** Kelimenin önüne gelir. “impossible” gibi. Türkçede ön ek çok azdır ve Arapça veya Farsçadan giren kelimelerde görülür. “natamam” tamamlanmamış, eksik demektir. “biçare” çaresiz demektir.
- **Son ek (suffix):** Gövdeyi takip eder. “yemeklerdekiler” gibi.
- **İç ek (infix):** Gövdenin içine eklenir. Filipince gibi dillerde daha sık görülür.
- **Çevre ek (circumfix):** Ekin bir parçası gövdenin önünde, bir parçası arkasında yer alır. Almanca “sagen” (söylemek) fiilinde “ge-” + “sag” + “-t” ile “gesagt” oluşur. Bu yapı kelimeyi ortaç (participle) hâline getirir.

  **Ders sonrası not:** Altyazıda Almanca son ek “ST” olarak geçiyor. “gesagt” kelimesinde son ek “-t”dir.

**Birleştirme (compounding).** Biçimbirimler birleşerek yeni kelimeler yapar. “dog” + “house”, köpek kulübesi demektir. “pazar” + “yer”, pazar yeri demektir. Anlam, parçalardan rahatça tahmin edilir.

**Çekim ve yapım ekleri.** Yapım ekleri (derivation) kelimenin anlamını, hatta türünü değiştirebilir. Çekim ekleri (inflection) anlamı değiştirmez, kişi veya zaman bilgisi verir.

**Klitikler (cliticization).** Kesme işaretiyle ayrılan İngilizce “’ve” gibi yapılar, Fransızcadaki “la”, “le”, Türkçede yeri geldiğinde ayrı yazılabilen “de”, “ki”, “mi” bu gruba girer.

**Ek sayısı.** Bir kelime birden fazla ek alabilir. Hocaya göre İngilizcede bu sayı en fazla 4-5’tir. Türkçe gibi sondan eklemeli (bitişken, agglutinative) dillerde çok daha fazla ek gelebilir. Örnek: “uygarlaştıramadıklarımızdanmışsınızcasına”. Günlük hayatta böyle kelime kullanılmaz ama biçimbilimsel analizde sorunsuz çözümlenir. İngilizcede birkaç kelimeyle, hatta bir cümleyle anlatılan şey Türkçede tek kelimeye sığabilir.

İngilizce ek örnekleri: “rewrites” kelimesinde “re-” ön ek, “-s” son ektir. “unbelievable” kelimesinde “un-” ön ek, “-able” son ektir.

Çözümleme çıktısındaki etiketlerin hepsini ezberlemek gerekmez, kullandıkça göz alışır. Ama çözümleme sonucu bir uygulamada girdi olarak kullanılacaksa hangi etiketin hangi bilgiyi verdiği bilinmelidir.

## 10. İngilizce ile Türkçe Arasındaki Fark

İngilizcede “car” ve “cars” kelimelerinin kökü kolayca bulunur. Kelimeye eklenen ek sayısı azdır ve ekler kökü bozmaz. Bu yüzden kökler bir sözlükte tutularak işlem hızlıca yapılır. Hoca, İngilizcede biçimbilimsel analizin bugün %100 başarılı denebileceğini söyledi. Yine de Python’daki bazı kök bulma fonksiyonlarının bazı kelimelere garip kökler verdiğini görmüş.

Türkçede ise durum daha zordur:

- Kurulan otomatlarda eksikler çıkabiliyor. Bir problemi çözmek için otomata yapılan ekleme başka birçok kelimeyi bozabiliyor. Hoca, on kelimeyi feda etmektense bir kelimenin yanlış çözümlenmesinin kabul edilebileceğini söyledi.
- Türkçe karmaşık bir biçimbilimsel yapıya sahiptir. Bir kelimenin bütün olası biçimlerini listelemek mümkün değildir.
- Tek bir kelime aldığı eklerle birçok anlamı üzerinde taşıyabilir.

**“You had been in the car.” ve “Arabadaymışsınız.”**

İngilizce cümle kelimelere ayrılır ve “car” kökü hemen bulunur. Türkçede ise tek bir kelimenin kökü bulunmalıdır. İnsan kökün “araba” olduğunu rahatça görür. Bilgisayara ise bunu kurallarla anlatmak gerekir.

Neden kök “ar” (utanma) veya “ara” değil de “araba” alınıyor? Çünkü çözümleyici adayları sırayla dener:

1. “ar” kök alınırsa, geri kalanların bu köke doğru sırada eklenebilecek ekler olması gerekir. Değilse kök “ar” değildir.
2. “ara” kök alınırsa, geri kalan “badaymışsınız” bu köke eklenebilmelidir. Eklenemiyorsa kök “ara” da değildir.
3. “araba” kök alınır. “-daymışsınız” kurallara uyan eklerdir. Kök “araba”dır.

Çözümleme: araba (isim) + “-da” (bulunma hâli) + “-ymış” (ek fiil, ismi eyleme dönüştürür) + “-sınız” (2. çoğul kişi).

**Biçim bilgisiyle çeviri.** Bu çözümleme çeviride de işe yarar: araba, “the car”; “-da”, “in”; “-ymış”, “had been”; “-sınız”, “you”. Basit cümlelerde bu çalışır. Daha karmaşık cümlelerde çok anlamsız çeviriler çıkar. Birden fazla anlama gelen kelimeler bunu daha da bozar. Hocaya göre geçmişte Türkçe-İngilizce çevirilerin başarısız olmasının sebebi buydu. Yüz binlerce Türkçe-İngilizce cümle çiftiyle eğitilen derin öğrenme modelleriyle çeviriler iyileşti.

## 11. Çoklu Çözüm ve Anlam Belirsizliği

Bir kelime her zaman tek bir şekilde çözümlenmez. “çocukları” kelimesi Zemberek gibi bir çözümleyicide birden çok çözüm verir:

- çocuk + çoğul + 3. tekil iyelik.
- çocuk + çoğul + belirtme hâli.
- çocuk + çoğul + 3. çoğul kişi iyeliği.
- Dördüncü bir çözüm.

Bunun sebebi, kelimenin cümlelerde farklı görevlerde kullanılabilmesidir:

- “Sevgi’nin çocukları Ayşe ve Mehmet geldiler.”
- “Yeni gelen çocukları gördünüz mü?”
- “Ayşe ile Mehmet’in çocukları Gökhan ile Sevgi’dir.”
- “Ayşe’nin çocukları Gökhan ile Sevgi’dir.”

Çözümleyici dört çözümü de verir ama o cümlede hangisinin doğru olduğunu söylemez. Türkçede en sık görülen çözüm ilk sırada gelir. Hoca, akademik çalışmalarda genelde ilk çözümün alındığını söyledi. Yalnız kelime sayılacaksa bu bir sorun değildir, çünkü her çözümde “çocuk” vardır. Ama anlamsal bilgi çıkarılacaksa doğru çözüm gerekir. O zaman biçimbilimsel analiz yetmez, sözdizimsel ve anlamsal analize de geçmek gerekir.

Hoca çözümleyici örneği olarak Zemberek ve NLTK’yı andı. Porter kök bulucusunun (Porter Stemmer) Türkçe için olmadığını, Türkçe için Snowball’un olabileceğini söyledi.

**“kalem” örneği.** İki çözüm vardır:

- Yalın hâl: “pencil” anlamındaki kalem. “Bu kalem yazmıyor.”
- “kale” + 1. tekil iyelik “-m”: “castle” anlamındaki kale. “Benim kalem yıkıldı.”

Çözümleyici ikisini de verir. İnsan hangisinin kastedildiğini cümleye bakarak anlar. Bilgisayar için kelimenin hangi kelimelerle kullanıldığı önemlidir: “kale” anlamındaki kelime “yazmak”

fiiliyle, “pencil” anlamındaki kelime “yıkılmak” fiiliyle kullanılmaz. Bu bir kelime anlamı belirsizliği (word sense ambiguity) problemidir.

## 12. Biçimbilimsel Analiz Nerede Kullanılır

**İnternette arama.** “Anadolu Hisarı konserleri başladı mı?” sorgusunda eskiden yalnız sabit kalıp (fixed pattern) aranırdı. Sonuçlarda yalnız “Anadolu Hisarı” yazan sayfalar gelirdi. Bugün “Anadolu Hisarı’nda”, “Anadolu Hisarı’nın konserleri” gibi ekli biçimler de getiriliyor.

**Yazım denetimi.** “Bunu mu demek istediniz?” önerisinde basit dizge eşleştirmenin yanında biçim bilgisine de bakılabilir.

**Bul ve değiştir (find and replace).**

- “Bütün oğulları kız yap” denirse ve metinde “Benim oğlum mükemmeldir.” varsa, “oğul” kalıbı “oğlum”u bulmaz. Doğru sonuç “Benim kızım mükemmeldir.” olmalıdır. “oğul”un “oğlum” biçimine dönüştüğü bilgisini biçimbilimsel analiz verir.
- “Bütün binaları ev yap” denirse “binadan” kelimesi “evdan” olur. Doğrusu “evden”dir.

**Makine çevirisi.** Önceki bölümdeki örnek bunu gösteriyor.

## 13. Bağlılık Analizi

> **Hocanın vurgusu:** Bağlılık analizi (dependency parsing) çok önemlidir. Kelimeler arasındaki bağlılıklar bilinirse 5N1K soruları çok rahat cevaplanır ve daha anlamsal ilişkiler çıkarılır.

Örnek cümle: “Bu okuldaki öğrencilerin en akıllısı şurada duran küçük kızdır.”

Biçimbilimsel analiz her kelimeyi etiketleriyle verir. Bu etiketler alanda çalışanların ortak bildiği etiketlerdir, ufak değişiklikler görülebilir:

| Etiket | Anlamı |
|---|---|
| Det | Belirleyici (determiner) |
| Noun | İsim |
| Adj | Sıfat |
| Adv | Zarf |
| Verb | Fiil |
| A3sg | 3. tekil kişi |
| A3pl | 3. çoğul kişi |
| Pnon | İyelik eki yok |
| Loc | Bulunma hâli (-de) |
| Gen | İlgi hâli, sahiplik |
| Nom | Yalın hâl |
| With | İsimden sıfat yapan “-li” |
| Zero | Ek almadan türetme |
| Pos | Olumlu |
| Pres | Şimdiki zaman |
| PresPart | Ortaç |

> **Ders sonrası not:** Hoca “Loc” etiketini anlatırken “yönelme” dedi. “Loc” (locative) bulunma hâlidir (-de). Yönelme hâli (-e) “Dat” (dative) ile gösterilir.

“okuldaki” kelimesi örnek olarak alınırsa: okul (isim) + “-da” + “-ki”. “-ki” eki kelimeyi sıfat hâline getirir.

Bu bilgiler yetmez. Bağlılık analizi kelimeler arasındaki ilişkileri ekler:

- “bu” → “okul”u belirtir.
- “okuldaki” → “öğrencileri” niteler.
- “öğrencilerin” → “akıllısı” ile sahiplik ilişkisi kurar.
- “en” → “akıllısı”nı niteler.

- “akıllısı” → “kızdır” ile özne ilişkisi kurar.
- “küçük” ve “duran” → “kız”ı niteler.
- “şurada” → “duran”ın yerini bildirir (kalma tümleci).

Bu ilişkiler de kurallar çerçevesinde çıkarılır, iyi dil bilgisi gerekir.

**Araçlar.** Hocanın bildiği kadarıyla Türkçe için açık kaynak bir bağlılık çözümleyicisi yok. Boğaziçi ve İTÜ’de, belki ODTÜ’de de akademik çalışmalar var. Ticari kazanç sağlanmayacağına dair bir belge imzalanırsa akademik amaçla paylaşılıyor. Hoca, bunların Türkçe üzerine çalışan firmalara satılabildiğini düşünüyor. İngilizce için Python kütüphaneleri çözümleme verir, hatta bağlılık ağacını çizer.

## 14. İngilizcede Çekim ve Yapım Ekleri

Çekim ekleri isimlere, fiillere ve sıfatlara uygulanır. İsimlerde çoğul ve sahiplik ekleri vardır.

**İsimler.** Düzenli (regular) ve düzensiz (irregular) isimler ayrı sözlüklerde tutulur. Düzenli isimlerin çoğul eki alması bellidir. Düzensizler ise çok değişkendir. “ox” ve “oxen” bilmeyen biri için iki ayrı kelime gibidir.

**Fiiller.**

- “walk”: walks, walking, walked.
- “try”: tries. “y” düşer, bu bir kuraldır.
- “stop”: stopping. Son harf tekrarlanır. Bu tür fiiller ayrı bir sözlükte tutulur.
- Düzensiz fiiller (“eat”, “ate”, “eaten”) ayrı bir sözlükte olmalıdır. Hocaya göre bunların aynı kökten geldiğini dili bilmeyen biri göremez.

**Yapım ekleri.** İngilizcede daha karışıktır, genellikle fiil ve sıfatlara uygulanır. “-ation”, “-ee”, “-er”, “-ness” ekleri fiil ve sıfatları isme dönüştürür (nominalization). Bazı ekler (“-less” gibi) isim ve fiillerden sıfat yapar.

## 15. Biçimbilimsel Çözümleyicinin Bileşenleri

Bir biçimbilimsel analiz için üç şey gerekir.

**1. Sözlük (lexicon).** İngilizcede bütün gövdeler konur. Türkçe gibi sondan eklemeli dillerde gövdeler ve alınabilecek ekler tutulur, ya da ekler için ayrı bir yapı kullanılır. Hece düşmesi, yumuşama, sertleşme gibi özel durumlar görülen kelimelere sözlükte bir bayrak (flag) konur. “Bu kelimede şu durum oluşabilir, kontrol et” anlamına gelir.

**2. Biçim dizimi kuralları (morphotactic rules).** Eklerin hangi sırayla eklenebileceğini söyler. “çocukların” kelimesinde önce çoğul eki, sonra sahiplik eki gelir. Sahiplik ekinden sonra çoğul eki gelmez: “çocukınlar” diye bir kelime yoktur.

**3. İmla kuralları (orthographic rules).** Türkçe için şunlar sayıldı:

- Ünlü uyumu kontrolü.
- **Ünlü düşmesi:** oğul → oğlu. “u” düşer.
- **Ünlü daralması:** anla + yor → anlıyor. “a”, “ı”ya daralır.
- **Ünsüz sertleşmesi:** meslek + daş → meslektaş.
- **Ünsüz yumuşaması:** ağaç + a → ağaca. “ç”, “c”ye dönüşür.
- **Ünsüz türemesi:** his + i → hissi.

  **Ders sonrası not:** Hoca ünsüz sertleşmesinde “D, T’ye, G, K’ya, Ç, C’ye dönüşüyor” dedi. Sertleşmede c, ç’ye dönüşür. Ç’nin c’ye dönüşmesi yumuşamadır. Hocanın hemen sonraki “ağaç” örneği de bunu gösterir.

**Bayrak kullanım örneği: “oğlu”.**

1. Ekler atılınca elde “oğl” kalır. Sözlükte böyle bir kelime yoktur.
2. Kural: Yan yana iki sessiz harf varsa ve arkasından “u” geliyorsa ünlü düşmesi olmuş olabilir. Bu sesli iki sessizin arasına konur: “oğul”.
3. “oğul” sözlükte vardır ve bu kuralın bayrağı açıktır. Kök “oğul” kabul edilir.

Bu tür kurallarla ters durumlardaki kelimelerin çözümünde ilerlenir.

## 16. İngilizce İçin Sonlu Durum Otomatları

Bu bölümdeki otomatlar kitaptan alınmıştır. Yalnız anlatmak içindir ve çok sınırlı sayıda kelime için çalışır. İngilizceyi bütünüyle çözmez.

**İsim otomatı.** Sözlükler: düzenli isimler, düzensiz çoğul isimler, düzensiz tekil isimler ve çoğul eki.

*"fox".* Başlangıçta üç yol vardır. Düzensiz tekil isim sözlüğünde yok. Düzensiz çoğul isim sözlüğünde yok. Düzenli isim sözlüğünde var. Girdi bitti ve bitiş durumundayız. Sonuç: "fox" isimdir.

*"foxing".* "fox" kısmına kadar düzenli isim sözlüğüyle gelinir, burası bitiş durumudur. Ama girdi bitmemiştir ve "-ing" ile gidilecek bir geçiş yoktur. Ya kelime İngilizce değildir ya da otomat eksiktir. Tamamlanmış bir otomatla çalışıldığı varsayılırsa kelime yanlış yazılmıştır ve çözümlenemez.

**Fiil otomatı.** Sözlükler: düzenli fiiller, düzensiz fiiller, düzensiz fiillerin geçmiş biçimleri. Ayrıca düzenli fiillerin alabileceği "-ed", "-ing", "-s" ekleri.

*"cuts".* "cut" düzenli fiil sözlüğünde yoktur, düzensiz fiil sözlüğünde vardır. Bitiş durumuna gelinir ama girdi bitmemiştir, "s" kalmıştır. "-s" ile başka bir bitiş durumuna geçilir. Girdi biter. Çıktı: "cut" düzensiz fiil + "-s".

> **Ders sonrası not:** Hoca buradaki "-s" için "çoğul takısı" dedi. Fiildeki "-s", geniş zamanda 3. tekil kişi ekidir.

Hoca, gerçek bir otomatın her ayrıntıyı düşünerek çok daha karmaşık hazırlanması gerektiğini vurguladı. Bu yüzden dil bilimcilerle çalışılır: Dil bilimci olası bütün durumları yazılı bir belgede verir, bu belge kodlanır. Kodlamadan sonra ancak otomatın çözemediği yeni bir kelimeyle karşılaşılınca dil bilimciye tekrar başvurulur.

**Sıfat otomatı: Eksik bir otomat ve düzeltmesi.**

İlk otomatta "bigger" tanınır: "un-" yoktur, ε ile sıfat kökü durumuna geçilir, "big" bulunur, "-er" ile devam edilir.

Ama İngilizcede olmayan "unbig" de tanınır: "un-" alınır, "big" sıfat sözlüğünde bulunur. Otomat bu problemi çözmeye yetmez.

Düzeltilmiş otomatta iki sıfat sözlüğü vardır:

- **Sıfat kökü 1:** "un-" ön ekini alabilen sıfatlar. Sonuna "-er", "-est", "-ly" alabilir.
- **Sıfat kökü 2:** "un-" almayan sıfatlar. Sonuna "-er", "-est" alabilir.

"unbig" için "un-" ile sıfat kökü 1'e gidilir ama "big" orada yoktur. ε ile diğer yola geçilirse "un" ile gidilecek yer yoktur. Kelime reddedilir. Yalnız "big" gelirse "un-" yolu kullanılmaz, sıfat kökü 2'de "big" bulunur.

Bu örnek, ek gruplarının ne kadar ince ayrıldığını gösterir. Bazı sıfatlar yalnız iki eki, bazıları üç eki alır. Bunlar çıkarılmazsa çözümleme yanlış olur.

**Türetme otomatı.** İsimden fiil ("-ize"), fiilden sıfat ("-able"), fiilden isim ("-ation") gibi geçişleri içerir. Farklı ek alan kelimeler için ayrı sözlükler vardır (fiil grubu j, fiil grubu k, isim grubu i gibi). İngilizcede türetme daha karmaşıktır. Bu otomatın da eksikleri vardır, kitaplar tamamını paylaşmaz.

Zemberek gibi araçların arkasında da Türkçe için çıkarılmış böyle bir otomat çalışır.

## 17. Kök Bulma: Soldan ve Sağdan

Köke varmak için iki yaklaşım vardır. Hoca, alandaki öncü çalışmayı yapan Kemal Oflazer'in çözümleyicisinin soldan başladığını söyledi. İTÜ'den Gülşen Hoca, Oflazer'in çalışmasını alıp iyileştirmiş; o da soldan gidiyor.

> **Ders sonrası not:** Altyazıda ad "Er Yiğit Cebiroğlu" olarak geçiyor. Doğrusu Gülşen Cebiroğlu Eryiğit'tir.

**Soldan yaklaşım, "okulda" örneği.**

1. “o”: Sözlükte var, zamirdir. Zamir otomatında geri kalan “kulda” bu köke eklenemez. Kök değil.
2. “ok”: Sözlükte var, isimdir. “-ulda” eklenemez, bitiş durumuna ulaşılmaz.
3. “oku”: Aynı kontrol yapılır.
4. “okul”: “-da” eklenebilir ve bitiş durumuna ulaşılır. Kök “okul”dur.

**Sağdan yaklaşım.** Türkçe sondan eklemeli olduğu için ekler hızla atılır.

1. “a” diye bir ek var. Atılır. “okuld” diye kök yok.
2. “da” eki olabilir. Atılır. “okul” köktür ve “-da” ona eklenebilir. Kök “okul”dur.

> **Hocanın vurgusu:** Hangi yönden gidilirse gidilsin, bir kökün kabul edilmesi için arkasından gelenlerin o köke eklenebilecek ekler olması ve otomatta bitiş durumuna ulaşılması gerekir.

**Basit bir otomat.** Hoca, Vasıf Nabiyev’in yapay zekâ kitabından alınan, sınırlı kelime için çalışan bir örnek gösterdi. Bir kelime isim kökünden gelmişse ekler şu sırayla gelir: çoğul, iyelik, durum, “-ki”. Hepsi olmak zorunda değildir ama varsa bu sırada olmalıdır. Fiil kökü için de benzer bir sıra vardır.

> **Ders sonrası not:** Altyazıda yazar adı “Vasit Mabiyev” olarak geçiyor. Doğrusu Vasıf Nabiyev’dir.

Gösterimde köşeli parantez seçenekleri gösterir: “D” yerine d veya t, “A” yerine a veya e gelir. Süslü parantez “olabilir de olmayabilir de” anlamındadır. Buradaki epsilon, hiçbir şey olmadan geçiş demektir. Bitiş durumları çift daireyle çizilir.

**“akıllardakilerin” örneği.**

1. “ak”: Sıfattır, isim sözlüğünde yok. Bir sonraki denenir.
2. “akı”: İsim sözlüğünde yok.
3. “akıl”: İsim sözlüğünde var. Geri kalanlar otomatta bitiş durumuna ulaşmalı.
4. “-lar”: Çoğul eki ile geçilir.
5. “-da”: Durum eki ile geçilir.
6. “-ki”: “-ki” eki ile geçilir.
7. “-ler”: Çoğul eki ile geçilir.
8. “-in”: İlgi eki ile geçilir. Girdi biter, bitiş durumuna ulaşılır. Kök “akıl”dır.

Bitirilemeseydi soldan bir harf daha eklenip yeni kök adayı denenecekti.

**Kök bulmadaki zorluklar.**

- **“yazıldığın”:** Kök “yaz” olarak da “yazı” olarak da alınabilir. Sonra hangi eklerin eklendiği bulunur.
- **“oğlumuz”:** Metinde “oğul” hiç geçmez. Ünlü düşmesi kuralıyla sesli harf araya konur, “oğul” sözlükte bulunur. Sonra “-umuz” ekinin bu köke eklenip eklenemeyeceğine bakılır.
- **“bacağım”:** Soldan da sağdan da gidilse “bacağ” ile “bacak” bulunamaz. Son harf “ğ” ise “k”nin yerine gelmiş olabilir. “k” konup sözlüğe bakılır, “bacak” bulunur. Sonra eklerin bu köke eklenip eklenemeyeceği kontrol edilir.

## 18. Türkçe Çözümleyicinin Akışı

Hoca bütün anlatılanları birleştiren bir akış şeması gösterdi ve bunu Oflazer ve Bozşahin’in çalışmasıyla ilişkilendirdi. Şemanın adımları sözlü anlatıma göre şöyledir:

```
1. Kelime gelir. Soldan veya sağdan bir kök adayı belirlenir.
2. Ünlü uyumu kontrol edilir. Uymazsa başka kök adayı alınır.
3. Kök fiilse fiil çözümleyicisine, değilse isim çözümleyicisine gönderilir.
   Yapım ekleriyle isimden fiile, fiilden isme geçilebilir.
4. Biçimbirim-ses (morphophonemic) denetiminde eklerin sırası kontrol edilir.
   Hata varsa adım 1'e dönülüp yeni kök belirlenir.
5. Bitiş durumuna ulaşılırsa çözüm verilir.
   Hiçbir yoldan ulaşılamazsa kelime bu dile ait değildir:
   ya yanlış yazılmıştır ya da başka bir şeydir.
```

Şemadaki her kutu kendi içinde bir otomattır.

## 19. Türkçe Ek Otomatları

Bu bölümdeki otomatlar, Gülşen Hoca'nın Türkçe için Oflazer'in çalışması üzerine geliştirip uyarladığı otomatlardır.

Ekler kelimelere rastgele eklenmez. Belirli kurallarla ve belirli bir sırayla eklenir. Hoca dört ek kümesi gösterdi:

- **İsim çekim ekleri:** 19 ek. Gösterimde büyük "H" harfi ı, i, u, ü yerine gelir. Örneğin "nHn" ile nın, nin, nun, nün ifade edilir.
- **Ek eylem ekleri:** 15 ek. İsimden eylem yapar.
- **Eylem zaman ekleri:** 29 ek.
- **Eylem çekim ekleri:** 44 ek.

**İsim çekim otomatı, "sandıktakilerden".** Kök "san" (sanmak) da olabilir, "sandık" da. Kelime isim kökünden gelmişse ekler otomatta dolaşır ve üç çıkıştan birine ulaşmalıdır.

1. "sandık" isim sözlüğünde bulunur.
2. Çoğul eki yok. Sıfır geçişle ilerlenir.
3. "-ta" aranıyor. Bu durumdan "-ta" ile gidilecek bir yer yok. İyelik eki de gelmediği için sıfır geçişle ilerlenir.
4. "-ta" durum ekidir, durum 1'e geçilir.
5. Sırada "-ki" var. Durum 1'den "-ki" ile gidilecek yer yok. Yanlış yol seçilmiş. Geri dönülür.
6. "-ta" ile gidilecek başka bir durum var: durum 2. Oraya geçilir.
7. "-ki" ile ilgi durumuna, "-ler" ile çoğul durumuna gidilir.
8. "-den" için doğrudan geçiş yok. Sıfır geçişle ilerlenir, "-den" ile durum 1'e geçilir.
9. Girdi biter, sıfır geçişle çıkışa ulaşılır. Kök "sandık"tır.

Bu örnek çözümleyicinin geri dönüşlü çalıştığını gösterir. Bir yol tıkanırsa başka yol denenir.

**Ek eylem otomatı, "akıllıymışsın".** "akıllı" isim soylu bir kelimedir. "-ymış" ile ek fiil durumuna, "-sın" ile kişi ekine geçilir. Sıfır geçişle çıkılır. Sonuç, eyleme dönüşmüş bir isimdir.

**Eylem zaman otomatı.** "gör" fiil kökü sözlükte bulunur. "-üyor" ile zaman durumuna, "-muş" ile bileşik zaman durumuna geçilir. "-sunuz" ile çıkışa gelinir.

**Olumsuzluk.** Otomatta bir olumsuzluk durumu da vardır ve önemlidir. "yapmazlarsa" kelimesindeki "-ma/-maz" olumsuzluk bildirir. Fiil kökü bu eki alarak yoluna devam edebilir.

**Eylem çekim otomatı, "görüşmezler".**

1. "gör" eylem köküdür.
2. "-üş" işteşlik ekidir, ilgili duruma geçilir.
3. Sıfır geçişlerle ilerlenir.
4. "-me" olumsuzluk ekiyle geçilir.
5. "-z" ve "-ler" ile çıkışa ulaşılır.

> **Hocanın vurgusu:** Türkçe için bu büyüklükte otomatları tek başına çıkarmak çok zordur. Yeni bir yapının otomata nereye ekleneceğine çok iyi karar verilmelidir. Yanlış eklenen bir dal başka kelimelerin yanlış çözümlenmesine yol açar. Dil bilimcilerle çalışılmasının sebebi budur.

## Dersin Toparlaması

Dersin ilk yarısında düzenli ifadeler, tür 3 düzenli dillerin pratik yazımı olarak ele alındı. Kleene operatörleri ve otomat karşılıklarıyla başlandı. Sonra `^`, `$`, `[ ]`, `[^ ]`, `.`, `?`, `*`, `+`, `\b`, `|` ve sayaçlar örneklerle işlendi. Bu az sayıdaki komut, arama, doğrulama, sayma ve metin işleme ihtiyaçlarının büyük bölümünü karşılar.

İkinci yarıda biçimbilimsel analiz anlatıldı. Kelime, biçimbirimlerine ayrılır. Bunun için sözlük, biçim dizimi kuralları ve imla kuralları gerekir. Çözümleme sonlu durum otomatlarıyla yapılır: Bir kök adayı seçilir ve geri kalanlar otomatta bitiş durumuna ulaşırsa kök kabul edilir. İngilizcede sözlük tabanlı yapı büyük ölçüde yeterlidir. Türkçede ise sondan eklemeli yapı ve ses olayları nedeniyle büyük otomatlar gerekir. Çözümleme tek başına cümledeki doğru anlamı vermez. Bunun için bağlılık analizi ve anlamsal analiz gerekir.

**Not:** İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.